
Experiment – 9

Student Name: Vikas

UID: 24BDA70164

Branch: Computer Science & Engineering

Section/Group: 24BDS-2(A)

Semester: 5th

Date of Performance: 17-09-26

Subject: Descriptive Analytics

Subject Code: 24CSH-344

1. Aim:

Analyze sales / customer segmentation data using Python (Pandas) and create an Excel (pivot-table-style) summary to identify how customers group by spending behaviour and city, so that the findings can support sales and marketing decisions.

2. Software Required:

- Visual Studio Code / Jupyter Notebook
- Python 3.8 or above
- Python libraries: pandas, numpy, matplotlib, openpyxl
- Microsoft Excel / Google Sheets (to view and verify the pivot table output)

3. Objective:

- To create a customer dataset containing Customer ID, Age, Annual Spend, Purchase Frequency, and City.
- To segment customers into “Budget”, “Standard”, “Premium”, and “Elite” categories using Pandas based on Annual Spend.
- To export the segmented data to a CSV file using Pandas.
- To build a pivot table in Excel (using openpyxl formulas) that summarizes the Count of customers and the Average Annual Spend, grouped by City (rows) and Spend Segment (columns).
- To visualize the distribution of customers across cities and spend segments using a stacked bar chart.
- To interpret the segmentation results and summarize the key findings.

4. Theory:

Customer Segmentation: the process of dividing customers into distinct groups based on shared characteristics – such as spending amount, age, purchase frequency, or location – so that businesses can target each group with relevant strategies. Segmentation is often performed by binning a continuous numeric variable (e.g., Annual Spend) into labelled categories.

Pandas cut(): a Pandas function used to bin continuous data into discrete intervals. It is used here to convert numeric Annual Spend values into the categorical labels Budget, Standard, Premium, and Elite.

Pivot Table: a data-summarization tool that reorganizes and aggregates data from a table, allowing values to be grouped by one or more categorical fields (rows and columns) and summarized using functions such as COUNT and AVERAGE. In Excel, a pivot table can be built manually using the PivotTable feature, or, as demonstrated in this experiment, replicated programmatically using COUNTIFS and AVERAGEIFS formulas so that the summary updates automatically whenever the source data changes.

Stacked Bar Chart: a variant of the bar chart in which each bar is divided into sub-segments representing the composition of a category, with sub-segment length proportional to its value. Unlike a simple bar chart, which shows only totals per category, a stacked bar chart shows both the total for each category (here, City) and how that total is composed of sub-groups (here, Spend Segment) – useful for comparing both overall volume and internal distribution at a glance.

5. Procedure/Algorithm:

Part A – Segmentation with Pandas:

- Import the pandas and numpy libraries.
- Generate a sample customer dataset of 120 customers (Customer_ID starting at 1001) with Customer_ID, Age, Annual_Spend, Purchase_Frequency, and City columns.
- Define spend bins (0–1500, 1500–3500, 3500–6000, 6000–9000) and corresponding labels (Budget, Standard, Premium, Elite).
- Use pd.cut() to create a new Spend_Segment column from Annual_Spend.
- Derive an additional Avg_Order_Value column ($\text{Annual_Spend} \div \text{Purchase_Frequency}$).
- Export the segmented DataFrame to customer_tiers.csv.

Part B – Pivot Table in Excel:

- Read the customer_tiers.csv file back into a DataFrame.
- Create a new Excel workbook using openpyxl; write the raw data to a “Data” sheet and format it as a styled table.
- Create a second “PivotTable” sheet with City as rows and Spend_Segment as columns.
- Use COUNTIFS to calculate the Count of Customer_ID and AVERAGEIFS to calculate the Average of Annual_Spend for every City × Segment combination, including Grand Total row and column.
- Apply formatting – header fills, currency format, borders, and frozen panes – for readability.
- Save the workbook as Customer_Tiers_PivotTable.xlsx.
- Plot a horizontal stacked bar chart of the number of customers per city, broken down by spend segment, using matplotlib and save it as a PNG image.

6. Code

Part A – Step-by-Step Python Code (Segmentation):

```
"""
PART A - Build a customer dataset with Pandas, segment it, export to CSV.

Differences from the earlier version:
* 120 customers instead of 100, IDs start at 1001
* City (5 values) instead of Region (4 values)
* Extra column: Purchase_Frequency (orders placed in the year)
* FOUR spend tiers instead of three: Budget / Standard / Premium / Elite
* Wider spend range (200 - 9000) and a different random seed
"""

import pandas as pd
import numpy as np

# -----
# Step 1: Create the sample customer dataset
# -----
np.random.seed(7)                # different seed -> different data
N_CUSTOMERS = 120

data = {
    "Customer_ID": range(1001, 1001 + N_CUSTOMERS),
    "Age": np.random.randint(21, 66, N_CUSTOMERS),
    "Annual_Spend": np.random.randint(200, 9000, N_CUSTOMERS),
    "Purchase_Frequency": np.random.randint(1, 40, N_CUSTOMERS),
    "City": np.random.choice(
        ["Delhi", "Mumbai", "Bengaluru", "Chennai", "Kolkata"], N_CUSTOMERS
    ),
}
df = pd.DataFrame(data)

# -----
# Step 2: Segment the customers into four spending tiers
# -----
bins = [0, 1500, 3500, 6000, 9000]
labels = ["Budget", "Standard", "Premium", "Elite"]
df["Spend_Segment"] = pd.cut(df["Annual_Spend"], bins=bins, labels=labels)

# A second derived column: how much each customer spends per order
df["Avg_Order_Value"] = (df["Annual_Spend"] / df["Purchase_Frequency"]).round(2)

# -----
# Step 3: Export to CSV
# -----
df.to_csv("customer_tiers.csv", index=False)

print("Data successfully processed and exported to CSV!")
print(df.head(10).to_string(index=False))
print("\nCustomers per segment:")
print(df["Spend_Segment"].value_counts().reindex(labels).to_string())
```

Part B – Step-by-Step Python Code (Pivot Table & Chart):

```
"""
PART B - Build a formatted Excel pivot table from the CSV, then draw the chart.

Differences from the earlier version:
* Pivot is TRANSPOSED: Rows = City, Columns = Spend_Segment
* Four segments and five cities
* Chart is a HORIZONTAL STACKED BAR chart (customers per city, split by
  segment) instead of a plain vertical bar chart of segment totals
"""

import pandas as pd
import matplotlib.pyplot as plt
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from openpyxl.worksheet.table import Table, TableStyleInfo

FONT = "Arial"
TEAL = "1F4E4A"
WHITE = "FFFFFF"
LIGHT_TEAL = "DCE9E6"
MID_TEAL = "BDD8D2"

df = pd.read_csv("customer_tiers.csv")
N = len(df)
LAST_ROW = N + 1

wb = Workbook()

# =====
# Sheet 1: Data (the CSV exported from Pandas)
# =====
ws = wb.active
ws.title = "Data"
cols = list(df.columns)
ws.append(cols)
for r in df.itertuples(index=False):
    ws.append(list(r))

header_fill = PatternFill("solid", fgColor=TEAL)
header_font = Font(name=FONT, bold=True, color=WHITE, size=10)
for c in range(1, len(cols) + 1):
    cell = ws.cell(row=1, column=c)
    cell.fill = header_fill
    cell.font = header_font
    cell.alignment = Alignment(horizontal="center", wrap_text=True)

for row in ws.iter_rows(min_row=2, max_row=LAST_ROW, min_col=1, max_col=len(cols)):
    for cell in row:
        cell.font = Font(name=FONT, size=10)
for r in range(2, LAST_ROW + 1):
    ws.cell(row=r, column=3).number_format = '$#,##0' # Annual_Spend
    ws.cell(row=r, column=7).number_format = '$#,##0.00' # Avg_Order_Value

widths = [12, 6, 14, 18, 12, 15, 16]
for i, w in enumerate(widths, start=1):
    ws.column_dimensions[get_column_letter(i)].width = w

tab = Table(displayName="CustomerData", ref=f"A1:{get_column_letter(len(cols))}{LAST_ROW}")
tab.tableStyleInfo = TableStyleInfo(name="TableStyleMedium7", showRowStripes=True)
ws.add_table(tab)
ws.freeze_panes = "A2"

DATA = "Data"
# Column order: A=Customer_ID, B=Age, C=Annual_Spend, D=Purchase_Frequency,
#               E=City, F=Spend_Segment, G=Avg_Order_Value
SPEND_COL = f"$C$2:$C${LAST_ROW}"
CITY_COL = f"$E$2:$E${LAST_ROW}"
SEG_COL = f"$F$2:$F${LAST_ROW}"

thin = Side(style="thin", color="BFBFBF")
border = Border(left=thin, right=thin, top=thin, bottom=thin)
```

```

# =====
# Sheet 2: PivotTable - Rows = City, Columns = Spend_Segment,
# Values = Count of Customer_ID AND Average of Annual_Spend
# =====
wsp = wb.create_sheet("PivotTable")
wsp["A1"] = "Pivot Table: Customer Count & Average Spend by City and Segment"
wsp["A1"].font = Font(name=FONT, bold=True, size=14, color=TEAL)
wsp["A2"] = ("Rows = City | Columns = Spend_Segment | "
            "Values = Count of Customer_ID, Average of Annual_Spend")
wsp["A2"].font = Font(name=FONT, italic=True, size=9, color="808080")

cities = ["Bengaluru", "Chennai", "Delhi", "Kolkata", "Mumbai"]
segments = ["Budget", "Standard", "Premium", "Elite"]

header_row1 = 4 # segment names (merged across 2 cols each)
header_row2 = 5 # "Count" / "Average Spend" sub-headers
first_data_row = 6

wsp.cell(row=header_row1, column=1, value="City")
wsp.merge_cells(start_row=header_row1, start_column=1, end_row=header_row2, end_column=1)

col = 2
seg_start_col = {}
for seg in segments:
    seg_start_col[seg] = col
    wsp.cell(row=header_row1, column=col, value=seg)
    wsp.merge_cells(start_row=header_row1, start_column=col,
                    end_row=header_row1, end_column=col + 1)
    wsp.cell(row=header_row2, column=col, value="Count of Customer_ID")
    wsp.cell(row=header_row2, column=col + 1, value="Average of Annual_Spend")
    col += 2

grand_total_start_col = col
wsp.cell(row=header_row1, column=col, value="Grand Total")
wsp.merge_cells(start_row=header_row1, start_column=col,
                end_row=header_row1, end_column=col + 1)
wsp.cell(row=header_row2, column=col, value="Count of Customer_ID")
wsp.cell(row=header_row2, column=col + 1, value="Average of Annual_Spend")

last_col = col + 1
for r in (header_row1, header_row2):
    for c in range(1, last_col + 1):
        cell = wsp.cell(row=r, column=c)
        cell.fill = header_fill
        cell.font = header_font
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)

# ---- one row per city -----
for i, city in enumerate(cities):
    r = first_data_row + i
    wsp.cell(row=r, column=1, value=city).font = Font(name=FONT, bold=True, size=10)
    wsp.cell(row=r, column=1).fill = PatternFill("solid", fgColor=LIGHT_TEAL)
    for seg in segments:
        c = seg_start_col[seg]
        cl = get_column_letter(c)
        wsp.cell(row=r, column=c,
                value=f'=COUNTIFS({DATA}!{CITY_COL},{A{r}},{DATA}!{SEG_COL},{cl}${header_row1})')
        wsp.cell(row=r, column=c + 1,
                value=(f'=IFERROR(AVERAGEIFS({DATA}!{SPEND_COL},{DATA}!{CITY_COL},{A{r}},'
                    f'{DATA}!{SEG_COL},{cl}${header_row1}),0)')
                ).number_format = '$#,##0')
        wsp.cell(row=r, column=grand_total_start_col,
                value=f'=COUNTIF({DATA}!{CITY_COL},{A{r}})')
        wsp.cell(row=r, column=grand_total_start_col + 1,
                value=f'=AVERAGEIFS({DATA}!{SPEND_COL},{DATA}!{CITY_COL},{A{r}}'
                ).number_format = '$#,##0')

# ---- grand total row -----
total_row = first_data_row + len(cities)
wsp.cell(row=total_row, column=1, value="Grand Total")
for seg in segments:
    c = seg_start_col[seg]
    cl = get_column_letter(c)

```

```

wsp.cell(row=total_row, column=c,
         value=f'=COUNTIF({DATA}!{SEG_COL},{cl}${header_row1})')
wsp.cell(row=total_row, column=c + 1,
         value=f'=AVERAGEIFS({DATA}!{SPEND_COL},{DATA}!{SEG_COL},{cl}${header_row1})'
         ).number_format = '$#,##0'
wsp.cell(row=total_row, column=grand_total_start_col, value=f'=COUNTA({DATA}!{SEG_COL})')
wsp.cell(row=total_row, column=grand_total_start_col + 1,
         value=f'=AVERAGE({DATA}!{SPEND_COL})').number_format = '$#,##0'

for c in range(1, last_col + 1):
    cell = wsp.cell(row=total_row, column=c)
    cell.font = Font(name=FONT, bold=True, size=10)
    cell.fill = PatternFill("solid", fgColor=MID_TEAL)

for r in range(header_row1, total_row + 1):
    for c in range(1, last_col + 1):
        wsp.cell(row=r, column=c).border = border
        if r >= first_data_row:
            wsp.cell(row=r, column=c).alignment = Alignment(horizontal="center")

wsp.column_dimensions["A"].width = 16
for c in range(2, last_col + 1):
    wsp.column_dimensions[get_column_letter(c)].width = 15
wsp.sheet_view.showGridLines = False

# Note the source of the numbers, as required for any generated workbook.
note_row = total_row + 2
wsp.cell(row=note_row, column=1,
         value=(f'Note: all figures are live formulas (COUNTIFS / AVERAGEIFS) reading the "'
                f"'Data' sheet. Source data is randomly generated in '"
                f"'part_a_pandas_segmentation.py (numpy seed = 7).'"'))
wsp.cell(row=note_row, column=1).font = Font(name=FONT, italic=True, size=9, color="808080")

wb.save("Customer_Tiers_PivotTable.xlsx")
print("Workbook saved.")

# =====
# Chart: HORIZONTAL STACKED BAR – customers per city, split by segment
# =====
matrix = (
    df.pivot_table(index="City", columns="Spend_Segment",
                   values="Customer_ID", aggfunc="count", fill_value=0)
    .reindex(index=cities, columns=segments, fill_value=0)
)

colors = {"Budget": "#9DC3E6", "Standard": "#70AD47",
          "Premium": "#FFC000", "Elite": "#C0504D"}

fig, ax = plt.subplots(figsize=(10, 6), dpi=150)
left = [0] * len(cities)

for seg in segments:
    values = matrix[seg].tolist()
    bars = ax.barh(cities, values, left=left, height=0.62,
                  color=colors[seg], edgecolor="#404040",
                  linewidth=0.7, label=seg)
    # label each block that is wide enough to hold text
    for bar, v, l in zip(bars, values, left):
        if v >= 2:
            ax.text(l + v / 2, bar.get_y() + bar.get_height() / 2, str(v),
                  ha="center", va="center", fontsize=9,
                  fontweight="bold", color="#1A1A1A")
    left = [l + v for l, v in zip(left, values)]

# total at the end of each bar
for y, total in enumerate(left):
    ax.text(total + 0.4, y, f'n = {total}', va="center",
          fontsize=9, fontweight="bold", color="#404040")

ax.set_title("Customer Spend Tiers by City (Stacked)",
            fontsize=14, fontweight="bold", pad=14)
ax.set_xlabel("Number of Customers", fontsize=11)
ax.set_ylabel("City", fontsize=11)
ax.set_xlim(0, max(left) * 1.16)

```

```

ax.grid(axis="x", linestyle="--", linewidth=0.7, alpha=0.5)
ax.set_axisbelow(True)
ax.invert_yaxis()
ax.legend(title="Spend Segment", loc="upper center", bbox_to_anchor=(0.5, -0.13),
          ncol=len(segments), frameon=False, fontsize=10)

fig.tight_layout()
fig.savefig("customer_tiers_stacked_bar.png", dpi=300, bbox_inches="tight")
print("Chart saved.")

```

7. Output:

Console Output (Part A):

```

PS C:\Users\Vikas\Desktop\files> & C:\Users\vikas\AppData\...
14-64\python.exe "c:/Users/vikas/Desktop/files/part_a_pandas...

Data successfully processed and exported to CSV!

Customer_ID  Age  Annual_Spend  Purchase_Frequency  City Spend_Segment  Avg_Order_Value
1001         25         8132             7  Mumbai         Elite         1161.71
1002         46         2438            32   Delhi        Standard          76.19
1003         24          926            30  Mumbai         Budget          30.87
1004         40         7380            17   Delhi         Elite         434.12
1005         44         4942             5 Bengaluru        Premium         988.40
1006         60         7856            39 Bengaluru         Elite         201.44
1007         49          643            14   Delhi         Budget          45.93
1008         35         8498             3 Bengaluru         Elite        2832.67
1009         44         7990            13  Kolkata         Elite         614.62
1010         29         6204            25  Kolkata         Elite         248.16

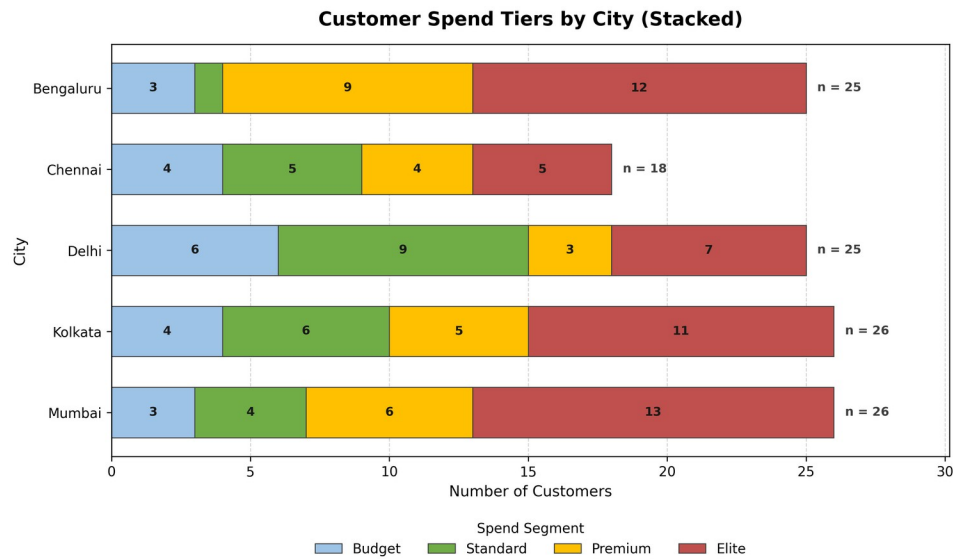
```

Excel Output – PivotTable sheet (Customer_Tiers_PivotTable.xlsx):

Pivot Table: Customer Count & Average Spend by City and Segment

City	Budget		Standard		Premium		Elite		Grand Total	
	Count of Customer_ID	Average of Annual_Spend	Count of Customer_ID	Average of Annual_Spend	Count of Customer_ID	Average of Annual_Spend	Count of Customer_ID	Average of Annual_Spend	Count of Customer_ID	Average of Annual_Spend
Bengaluru	3	\$783	1	\$2,249	9	\$4,528	12	\$7,585	25	\$5,455
Chennai	4	\$534	5	\$1,874	4	\$4,858	5	\$7,556	18	\$3,817
Delhi	6	\$815	9	\$2,551	3	\$4,106	7	\$7,750	25	\$3,777
Kolkata	4	\$850	6	\$2,316	5	\$4,474	11	\$7,356	26	\$4,638
Mumbai	3	\$802	4	\$2,405	6	\$4,704	13	\$7,108	26	\$5,102
Grand Total	20	\$759	25	\$2,324	27	\$4,559	48	\$7,424	120	\$4,606

Stacked Bar Chart – Customer Spend Tiers by City:



8. Conclusion:

Customer segmentation was successfully performed on the sample dataset of 120 customers using Pandas. Based on Annual Spend, 20 customers were classified as Budget, 25 as Standard, 27 as Premium, and 48 as Elite. The Excel pivot table, built programmatically with COUNTIFS and AVERAGEIFS formulas, further broke this down by City: Mumbai and Kolkata had the largest customer bases (26 customers each), while every city's largest single group was the Elite tier – driven by the wide \$200–\$9,000 spend range – and Delhi recorded the highest Elite-tier spend on average (\$7,750).

9. Learning Outcomes:

- Learned to create and prepare a customer dataset using Pandas.
- Learned to segment numeric data into categories using `pd.cut()`.
- Learned to export processed data from Pandas to a CSV file.
- Learned to format Excel worksheets professionally (headers, fills, borders, number formats, frozen panes).
- Learned to build a two-way (City × Segment) pivot table with live COUNTIFS/AVERAGEIFS formulas.
- Learned to visualize multi-category summary data using a matplotlib stacked bar chart.